



Introducción a **Python**
para la **Ciencia Abierta**



CSIC
2026

GUSTAVO.LINAN@CSIC.ES

IMSE
-cnm



Instituto de
Microelectrónica
de Sevilla

Nos presentamos...



Módulo 1:

Introducción



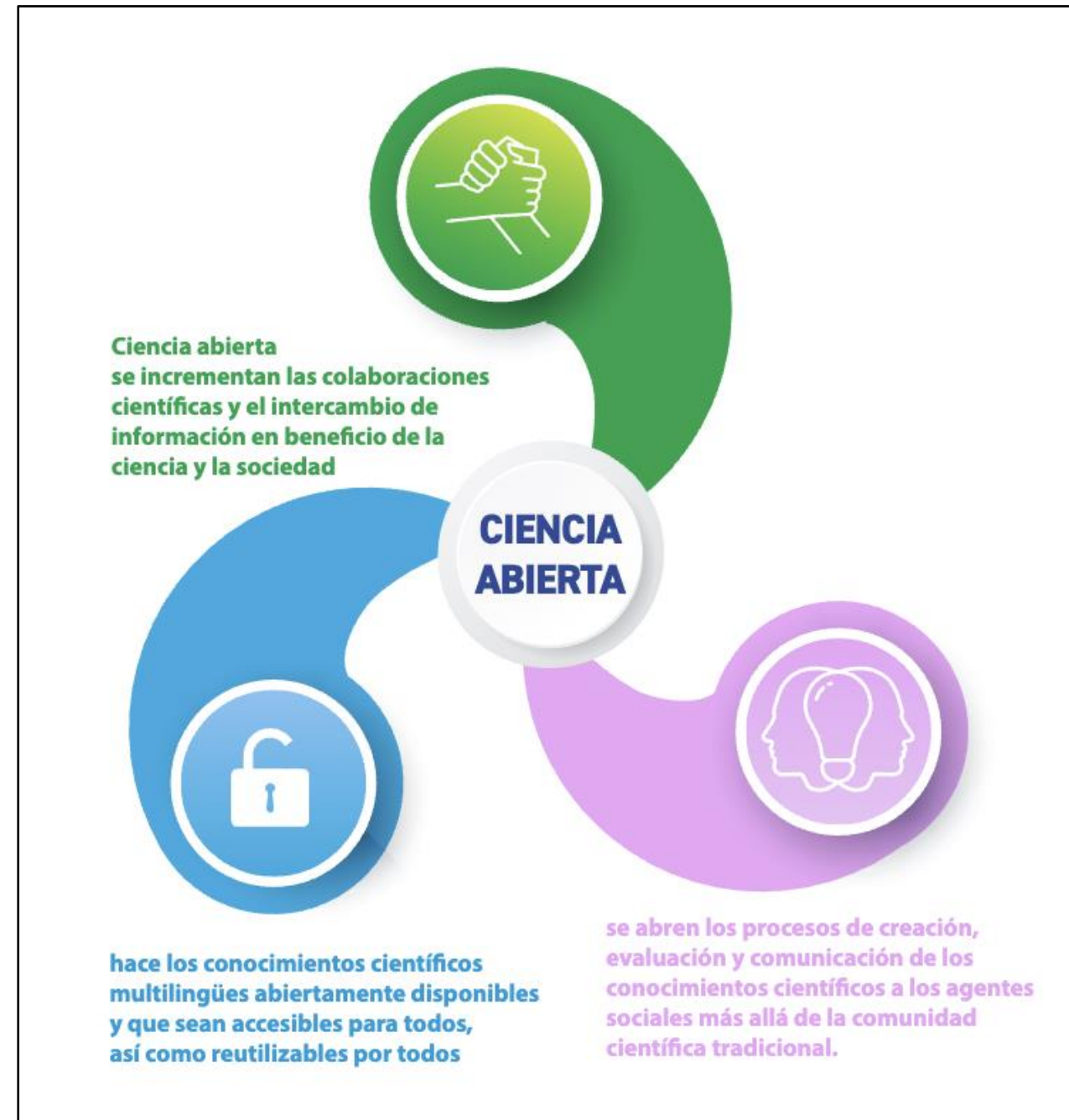
OBJETIVOS:

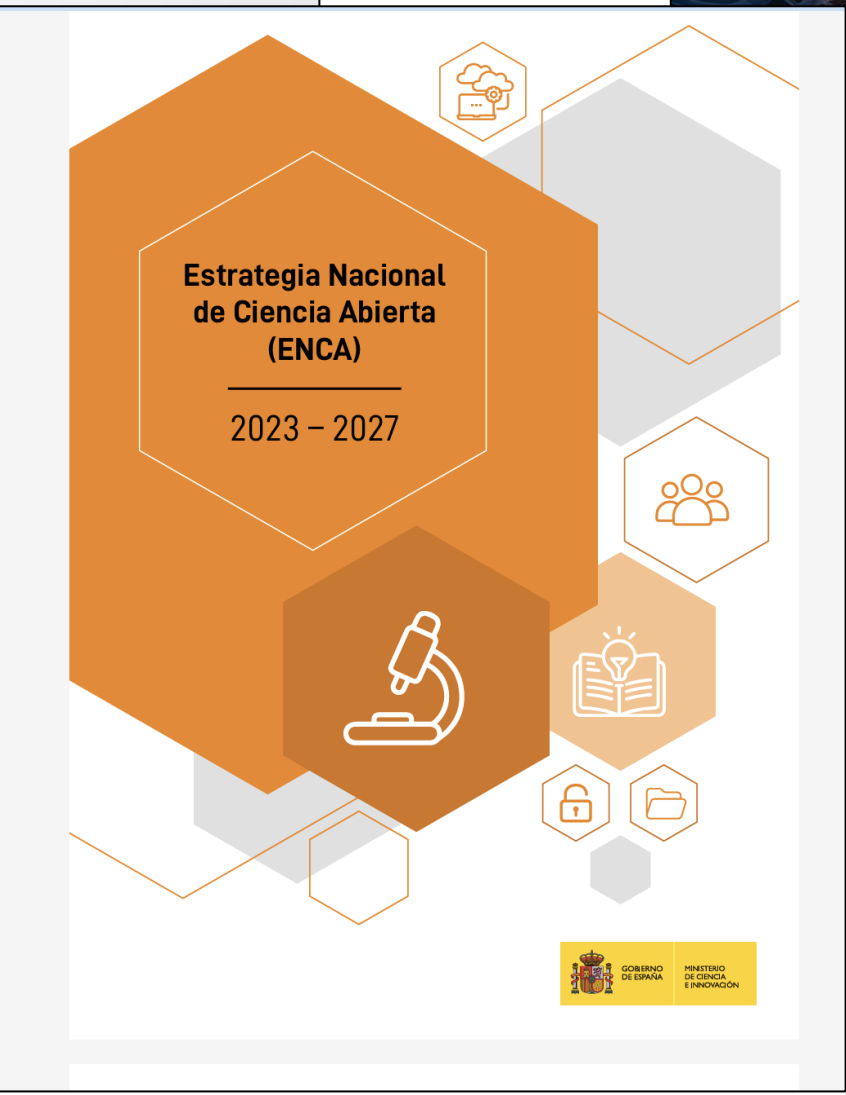
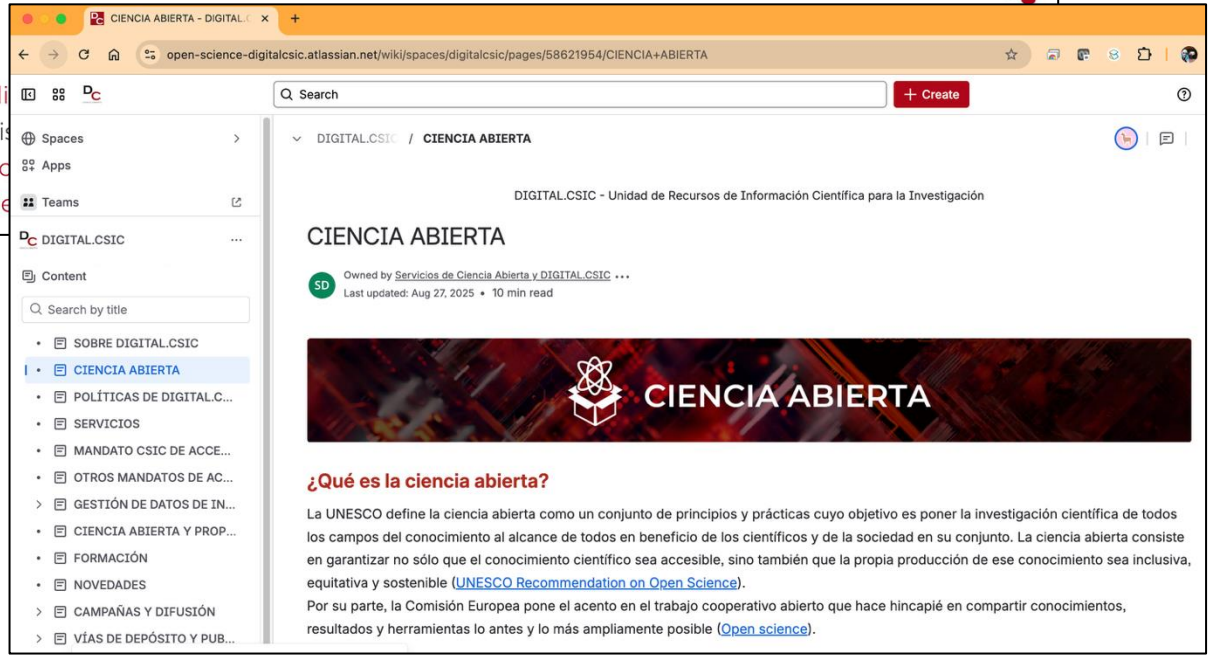
1. Entender qué es Python y sus aplicaciones en la ciencia abierta.
2. Instalar y configurar Python
3. Instalar y configurar Antigravity Crear y gestionar entornos virtuales.
4. Comprender el uso de requirements.txt para gestionar dependencias.



“La ciencia abierta es un constructor inclusivo que combina diversos movimientos y prácticas con el fin de que los **conocimientos científicos** multilingües estén **abiertamente disponibles** y **sean accesibles** para todos, así como **reutilizables** por todos, se **incrementen las colaboraciones** científicas y el **intercambio** de información en beneficio de la ciencia y la sociedad, y **se abran los procesos de creación, evaluación y comunicación** de los conocimientos científicos a los agentes sociales **más allá de la comunidad científica tradicional.**”

Recomendación de la UNESCO sobre CIENCIA ABIERTA, 2021,
<https://doi.org/10.54677/YDOG4702>



CIENCIA ABIERTA - DIGITAL.CSIC

open-science-digitalcsic.atlassian.net/wiki/spaces/digitalcsic/pages/58621954/CIENCIA+ABIERTA

Search

Spaces Apps Teams DIGITAL.CSIC Content

Search by title

- SOBRE DIGITAL.CSIC
- CIENCIA ABIERTA**
- POLÍTICAS DE DIGITAL.C...
- SERVICIOS
- MANDATO CSIC DE ACCE...
- OTROS MANDATOS DE AC...
- GESTIÓN DE DATOS DE IN...
- CIENCIA ABIERTA Y PROP...
- FORMACIÓN
- NOVEDADES
- CAMPAÑAS Y DIFUSIÓN
- VÍAS DE DEPÓSITO Y PUB...

DIGITAL.CSIC - Unidad de Recursos de Información Científica para la Investigación

CIENCIA ABIERTA

Owned by Servicios de Ciencia Abierta y DIGITAL.CSIC ...

Last updated: Aug 27, 2025 • 10 min read

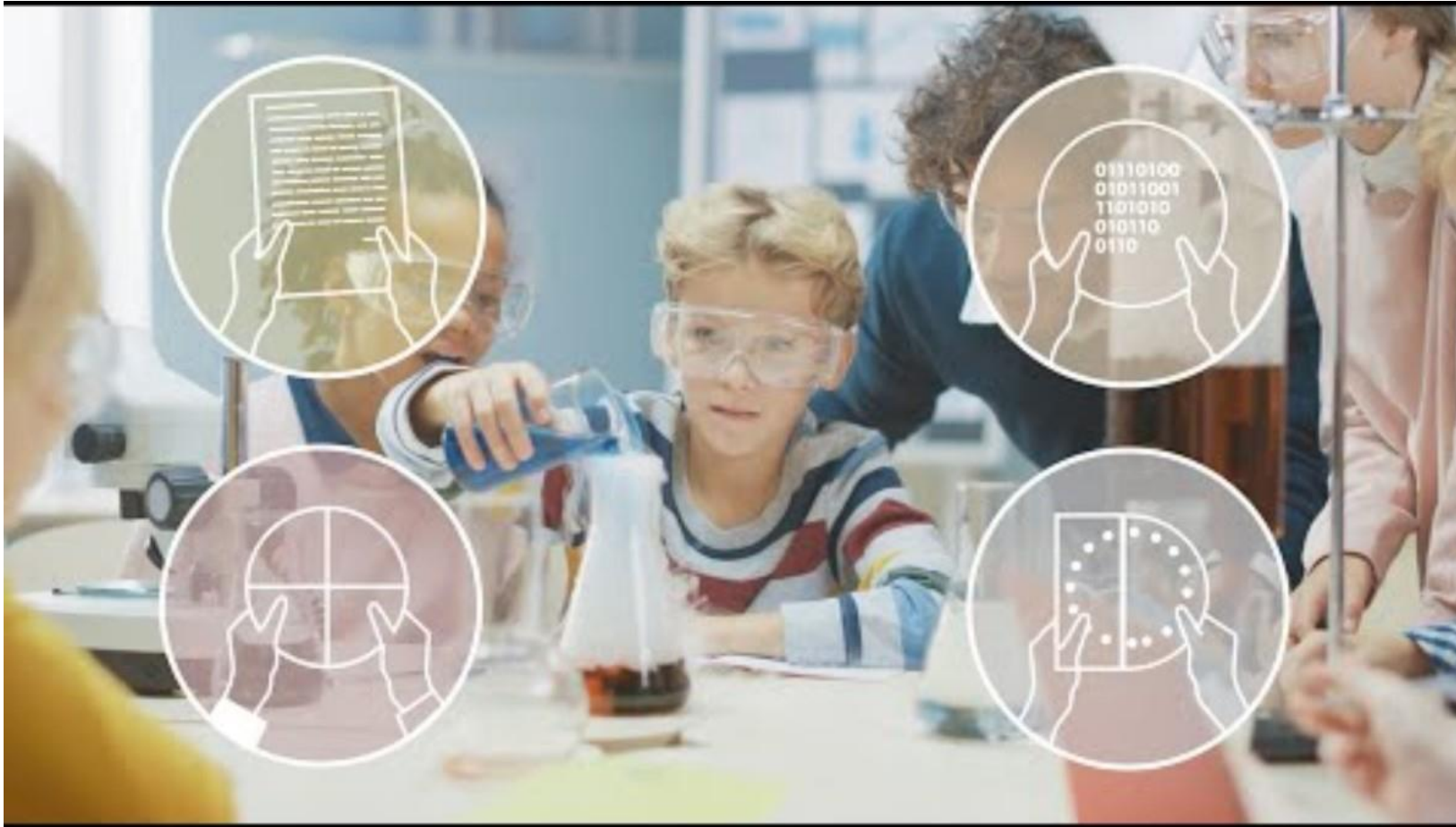


¿Qué es la ciencia abierta?

La UNESCO define la ciencia abierta como un conjunto de principios y prácticas cuyo objetivo es poner la investigación científica de todos los campos del conocimiento al alcance de todos en beneficio de los científicos y de la sociedad en su conjunto. La ciencia abierta consiste en garantizar no sólo que el conocimiento científico sea accesible, sino también que la propia producción de ese conocimiento sea inclusiva, equitativa y sostenible ([UNESCO Recommendation on Open Science](#)).

Por su parte, la Comisión Europea pone el acento en el trabajo cooperativo abierto que hace hincapié en compartir conocimientos, resultados y herramientas lo antes y lo más ampliamente posible ([Open science](#)).

CIENCIA ABIERTA



Introducción a Python



Introducción a Python



El creador del lenguaje, **Guido van Rossum**, eligió el nombre "**Python**" en honor al grupo de comedia británico **Monty Python**, específicamente por su programa *Monty Python's Flying Circus*. Van Rossum quería un nombre corto, único y algo misterioso, y como era fan del humor absurdo del grupo, pensó que sería perfecto.

Así que el nombre refleja más el espíritu ligero y accesible del lenguaje que cualquier relación con serpientes

Introducción a Python



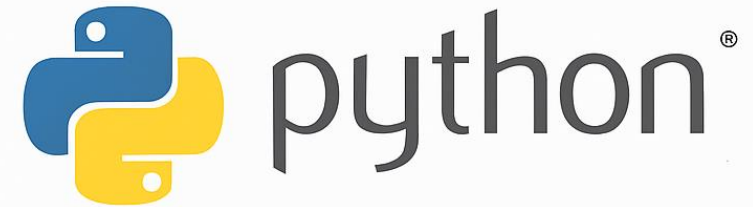
En uno de sus sketches más famosos, una pareja entra a un restaurante donde todos los platos contienen **Spam (carne enlatada)**, y los camareros repiten la palabra constantemente, interrumpiendo todo lo que dicen los clientes. La palabra "Spam" se convierte en sinónimo de algo **repetitivo, molesto y omnipresente**.

Este uso fue adoptado por la comunidad informática en los años 80 y 90 para describir mensajes no deseados que saturaban los canales de comunicación, como los correos electrónicos o los foros.

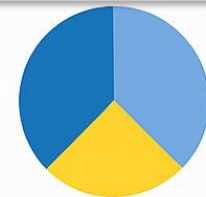
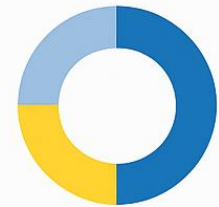
Introducción a Python

Python

- Python es un lenguaje de programación interpretado, de alto nivel y multiparadigma.
- Fue creado por Guido van Rossum en 1991 y ha ganado popularidad por su simplicidad y legibilidad.
- Tiene una comunidad activa y una gran cantidad de librerías disponibles.



Librerías
disponibles



Introducción a Python

Popularidad: Python es el lenguaje más popular en el índice TIOBE con un 25.35% de cuota, superando a todos los demás

En la **Stack Overflow Survey 2025** Python fue usado por **57,9%** de todos los encuestados.

Comunidad: estimaciones basadas en **SlashData** sitúan la comunidad de Python en torno a **~18 millones** a mediados de **2024**.

En la encuesta **PSF/JetBrains 2024**, para la pregunta “**Python como lenguaje principal**” por ámbito, los valores son aprox.: **Análisis de datos 49%**, **Desarrollo web 48%**, **Machine Learning 42%**, **Data Engineering 33%**

ECOSISTEMA DE LIBRERIAS COMUNES EN CIENCIA ABIERTA



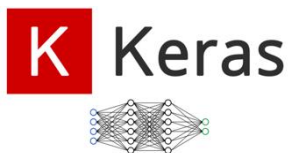
Computación científica y manejo de arrays.



Análisis y manipulación de datos.



Algoritmos clásicos de machine learning.



Redes neuronales y aprendizaje profundo.



Desarrollo de APIs modernas y rápidas.



Procesamiento de datos a gran escala con alto rendimiento.

Introducción a Python

Ámbitos de aplicación:

- Ciencia de datos: Análisis estadístico, visualización y modelado.
- Machine Learning: Clasificación, regresión, clustering.
- Desarrollo web: Backend con frameworks como Django y FastAPI.
- Automatización: Scripts para tareas repetitivas.
- Educación: Lenguaje ideal para enseñar programación.

Características de Python

Interpretable

Códigos fáciles
de leer

Tipado
dinámico

Multiparadigma

Gran
ecosistema de
librerías

Características de Python: Interpretable

- Qué significa?



PYTHON SE EJECUTA LÍNEA
POR LÍNEA SIN NECESIDAD
DE COMPILACIÓN PREVIA.



EJEMPLO: EJECUTA
DIRECTAMENTE: PYTHON
SCRIPT.PY

Características de Python:

Sintaxis clara y legible

¿Qué significa?

Python fue diseñado con la filosofía de que el **código debe ser tan legible como el lenguaje natural**.

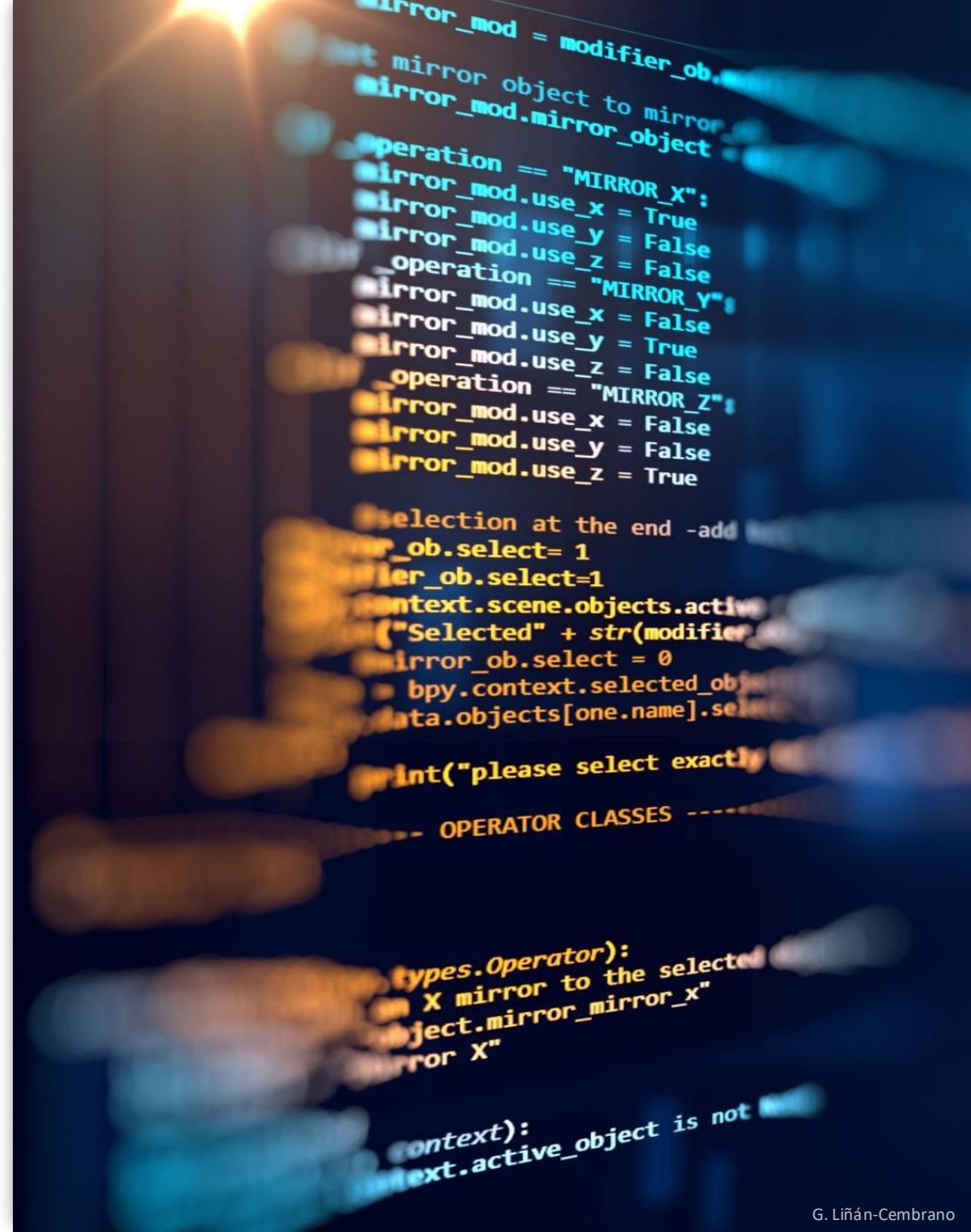
Esto facilita su aprendizaje, mantenimiento y colaboración entre desarrolladores.

🔍 Principios clave:

Indentación obligatoria: estructura el código de forma visual y jerárquica.

Palabras clave intuitivas: como if, for, def, class.

Evita el uso excesivo de símbolos {} () [] para delimitar bloques



Características de Python: Tipado dinámico

- Qué significa?
- No es necesario declarar tipos; Se "deducen" en tiempo de ejecución.
- Ejemplo:

`x = 'Hola' # x sería una cadena de texto`

`x = 37 # ahora x sería un numero`

`x = True # ahora x sería un valor lógico`



Características de Python: Multiparadigma

- Qué significa?
- Python permite escribir programas utilizando distintos estilos de programación, lo que lo hace extremadamente **flexible y adaptable** a diferentes tipos de problemas y preferencias del desarrollador.

Características de Python: Multiparadigma

Programación estructurada:

Uso de estructuras de control como bucles, condicionales y funciones.

```
1 for i in range(5):
2     print(i)
3
```

Características de Python:

Multiparadigma

Programación Orientada a Objetos (OOP)
Permite modelar entidades complejas mediante clases con sus atributos y métodos, y crear objetos pertenecientes a tales clases.

```
class Persona:
```

```
    def __init__(self, nombre, edad, profesion):
```

```
        self.nombre = nombre
```

```
        self.edad = edad
```

```
        self.profesion = profesion
```

```
        self.amigos = []
```

```
    def saludar(self):
```

```
        return f"Hola, soy {self.nombre} y tengo {self.edad} años."
```

```
    def trabajar(self):
```

```
        return f"{self.nombre} está trabajando como {self.profesion}."
```

```
    def agregar_amigo(self, amigo):
```

```
        self.amigos.append(amigo)
```

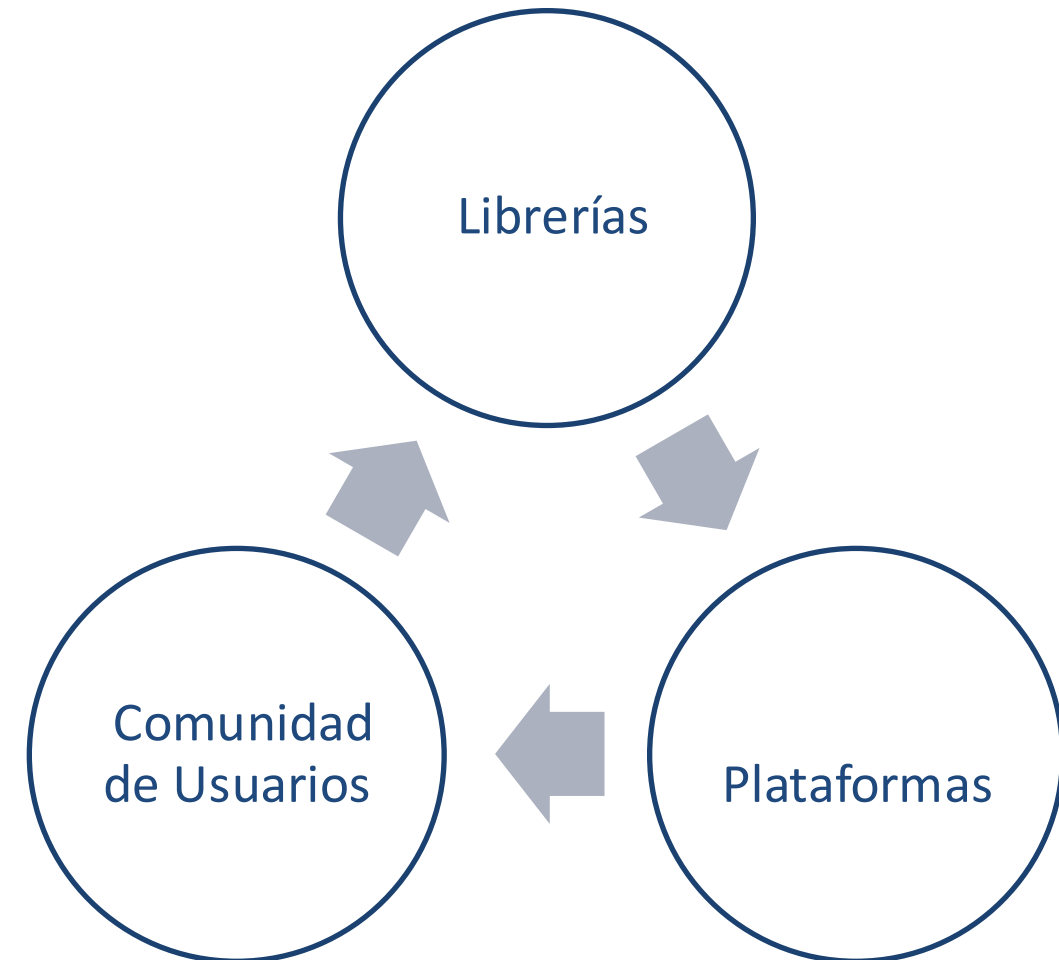
```
        return f"{amigo} ha sido añadido a la lista de amigos de {self.nombre}"
```

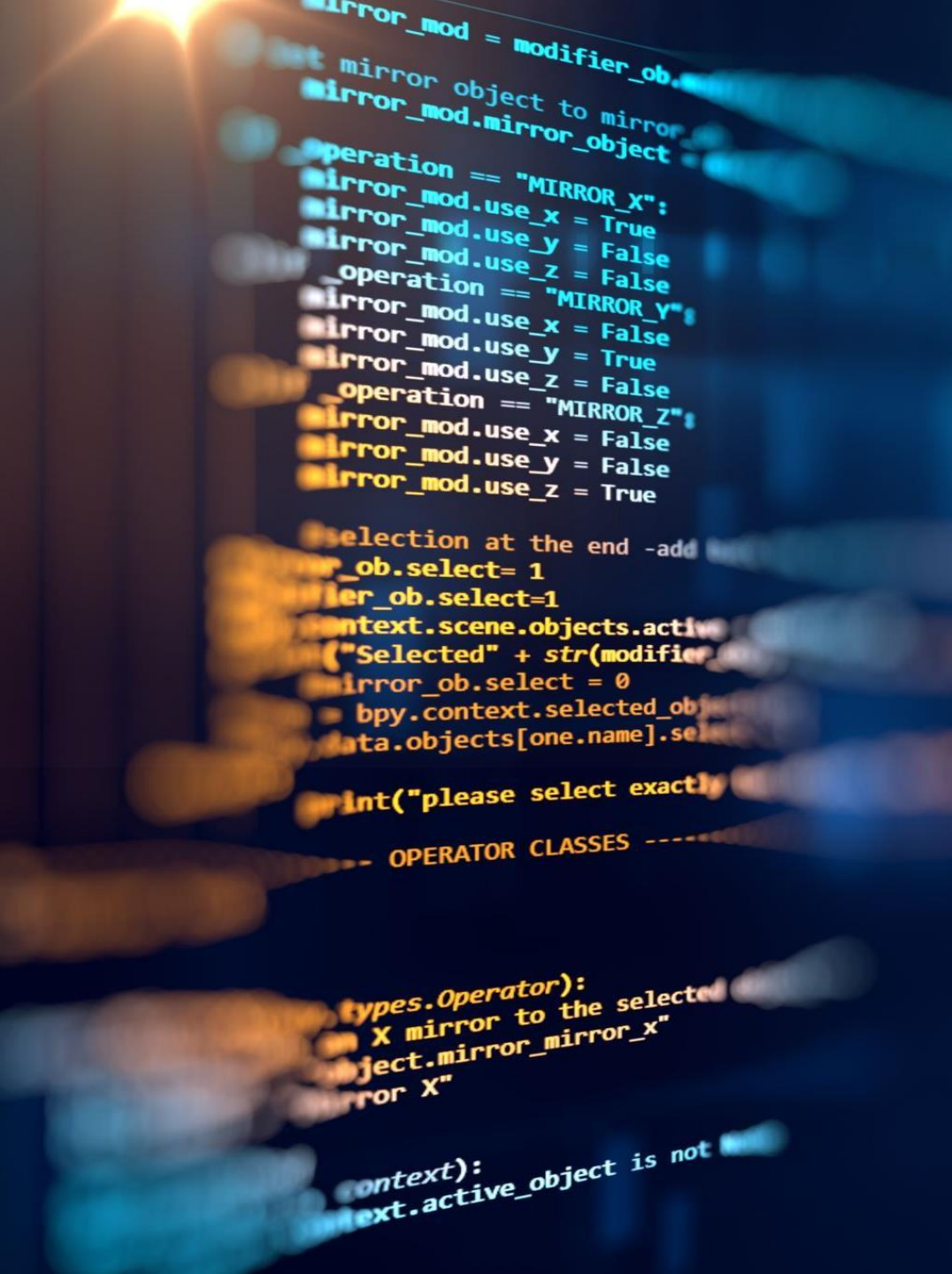
```
pepe = Persona("Pepe", 45, "Médico"); pepe.amigos = ["Juan", "Mari"]
```



Características de Python: Gran Ecosistema:

Gran Ecosistema de Python





Características de Python: Portabilidad

¿Qué significa?

- El mismo código (o casi...) te va a funcionar en Windows, MacOS, o Linux. Facilita la colaboración en equipos diversos.
- Reduce problemas de compatibilidad.
- Ideal para entornos científicos y educativos.



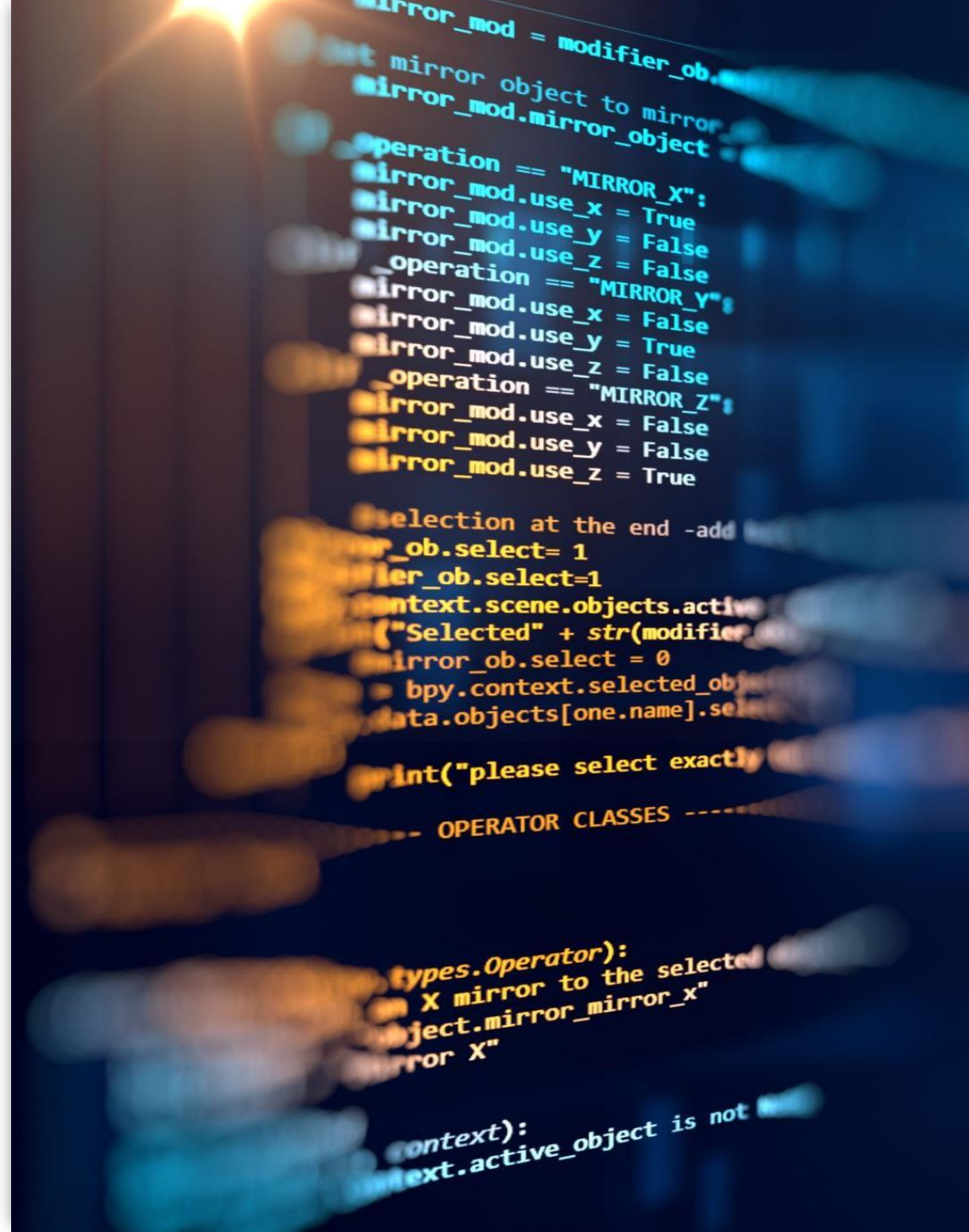
Python en la Ciencia Abierta

¿Por qué?

Facilita la compartición de código y resultados.

Promueve la reproducibilidad.

Compatible con plataformas abiertas como Jupyter y GitHub.



Python en la Ciencia Abierta

¿Por qué?

Facilita la compartición de código y resultados.

Promueve la reproducibilidad.

Compatible con plataformas abiertas como Jupyter y GitHub.

The screenshot shows the GitHub profile of Gustavo Liñán cembrano. The profile picture is a circular image of a hand holding a small, dark-colored bird. The name 'Gustavo Liñán cembrano' is displayed below the picture, along with the username 'guslicem · he/him'. The page indicates '310 contributions in the last year' and shows a calendar heatmap for the year 2026. The 'Popular repositories' section lists 'AISAD' as a public repository with 3 stars and 1 fork. The page also includes navigation tabs for Overview, Repositories (15), Projects, Packages, and Stars (4).

¿Qué es un entorno virtual?

Un entorno virtual es una carpeta aislada que contiene una instalación específica de Python y sus paquetes. Permite ejecutar proyectos con sus propias dependencias sin interferir con otros proyectos o con el sistema global.

- **Aislamiento** de dependencias.
- **Evita conflictos** entre proyectos.
- **Facilita la reproducibilidad.**

Creación y activación de entornos virtuales

- Crear:
`python -m venv nombre_entorno`
- Activar:
`source nombre_entorno/bin/activate`



`nombre_entorno\Scripts\activate`



Uso de pip y requirements.txt

pip es el gestor de paquetes oficial de Python. Permite instalar, actualizar y desinstalar librerías y herramientas desde el repositorio PyPI (Python Package Index), que contiene más de 400.000 paquetes disponibles.

Instalar paquetes:

pip install nombre_paquete

Desinstalar un paquete:

pip uninstall nombre_paquete

Crear archivo de dependencias con todos los paquetes que tenemos instalados en el entorno activo:

pip freeze > requirements.txt

Instalar desde archivo de dependencias:

pip install -r requirements.txt

Desinstalar todos los paquetes listados en requirements.txt:

pip uninstall -r requirements.txt -y

Actividad Práctica: Entornos virtuales y dependencias

Crear, activar,
desactivar un
entorno virtual



Instalar
dependencias



Crear y usar un
archivo de
dependencias

Actividad Práctica: Exploramos un código

- Revisar ejemplos de código en Python.
- Identificar características del lenguaje.
- Discusión abierta sobre casos reales en ciencia abierta.



Ideas Clave

¿Qué es Python y por qué usarlo?

Lenguaje versátil, legible y con gran comunidad.

Ideal para ciencia abierta: reproducibilidad, colaboración y transparencia.

Características clave de Python

Interpretable, tipado dinámico, multiparadigma.

Ecosistema rico en librerías científicas
(NumPy, Pandas, Scikit-learn, etc.).

Portabilidad entre sistemas operativos.

Entorno de desarrollo recomendado

Visual Studio Code con extensiones para Python y Jupyter.

Configuración personalizada para facilitar el trabajo científico.

Buenas prácticas desde el inicio

Uso de entornos virtuales para aislar dependencias.

Gestión de paquetes con pip y requirements.txt.

Actividades prácticas realizadas

Instalación de Python y Google Antigravity IDE.

Creación y activación de entornos virtuales.

Exploración de código real aplicado a ciencia abierta.